上海交通大学
SHANGHAI JIAO TONG UNIVERSITY

SJTU



互联网产品开发与运维

任锐

2026/10/27

饮 水 思 源 • 爱 国 荣 校

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

资源管理：伸缩

模块 6 · 资源管理（3/3）

饮 水 思 源 • 爱 国 荣 校

SJTU

本讲位置：从「放哪儿」到「何时变」

上一讲 · L-10

放置方案：3 个节点、6 个 Tomcat，坏一个节点仍然够用

遗留问题：平时也开着这么多，多 50%（L-10 第 27 页）

本讲问题

流量变了，实例数怎么跟着变，才能既不超时也不浪费？

引例（设定）

平时缩到 2 个节点、2 个 Tomcat，开放日交给按 CPU 的自动伸缩。读流量 30 秒内从 100 涨到 2400 QPS，新实例 9:01:35 才就绪（**症状 A：赶不上**）；把目标调低、上限调高后扩到 7 个，提交报 Too many connections（**症状 B：扩错层**）

本讲结构

1 扩哪一层

2 何时变

3 怎么变

4 两层伸缩

5 课程实践：弹性方案

课程实践以一张「弹性方案」回答引例

本模块 资源管理（L-09 ~ L-11）：隔离与分配（L-09）· 调度（L-10）· 伸缩（本讲）

来源：L-10 第 27 页；引例中的数字均为设定值或推算

伸缩的原问题

回顾思考题 1 第三幕

「平时 4 台机器只用 3%，能不能关掉两台省电，选课前再开？」

当时回答的是「关掉」需要哪些前提；本讲回答「怎么开、什么时候开」

扩哪一层	瓶颈在不在被扩的那一层 · 第一节
何时变	提前，还是跟着指标 · 第二节
怎么变	按什么信号、扩多少、缩多快 · 第三节

两个前提（已讲过）

实例无状态：状态在实例之外（L-05 第 25 页）

启动足够快：启动时间决定扩容的响应速度（L-05 第 24 页）

钱怎么省

容器编排按节点池计费（L-10 第 19 页）

实例缩了、节点没缩，钱没省（第 17 页）

要点 伸缩不是「多开几台」，而是在对的层、对的时间，让容量跟上需求

来源：思考题 1 第三幕；L-05 第 24、25 页；L-10 第 19 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

一、扩哪一层

第十一讲 · 伸缩

饮 水 思 源 • 爱 国 荣 校

SJTU

扩哪一层 • 瓶颈

先问瓶颈在哪一层

层	能否靠加实例扩容	前提或限制	选课平台的做法
Nginx（入口）	能，本讲不展开	主备加虚拟 IP（Q-1 第四幕）；再往上见 L-04 第六、七阶段	不自动伸缩
Tomcat（无状态）	**能**	状态在实例之外（L-05 第 25 页）；瓶颈确实在这一层	**本讲的伸缩对象**
批处理	不需要	可重跑、可被挤压（L-09 第 7 页）	填空闲、低优先级
MySQL 写	**不能**	有状态；上限是磁盘 fsync，约 250 TPS（L-07 第 25 页）	限流、组提交（实验 2）、Redis 原子扣减名额（思考题 2）
MySQL 读	能，但慢	加从库要搬数据、建复制，以小时计	提前准备，不进自动伸缩

判断方法 先用 L-08 的指标找到饱和的资源（USE），再看它所在的层能不能水平扩

回顾思考题 1 第五幕：加到 16 台，提交 TPS 只到 1010，数据库 CPU 95%

要点 伸缩只对「瓶颈在被扩的层、且这层无状态」有效；否则加得越快，下游越早被压垮

来源：L-05、L-07、L-08 讲义；思考题 1 第四、五幕

资源管理 – 伸缩

扩哪一层 • 下游

扩上游会压垮下游

连接数	每个 Tomcat 有自己的连接池：Σ连接池 + 其他连接 ≤ max_connections（MySQL 默认 151，L-06 第 19 页）。HikariCP 默认是固定大小的池，实例一启动就建满
锁与 fsync	写的上限在磁盘（L-07 第 25 页）；更多 Tomcat 只带来更多并发事务，热点行的锁等待更长（L-06 第 20 页）
在途请求	下游变慢 → 每个请求停留更久 → 在途请求增多 → 按在途请求伸缩时，伸缩器会继续加 Tomcat（第 14 页）

问题	原因	对策
扩容后报 Too many connections	Σ连接池超过 max_connections	伸缩上限按数据库的连接预算定
扩容后提交 p99 更差	瓶颈在磁盘，扩的却是上游	写入口按磁盘能力限流（L-09 第 22 页）
下游变慢，伸缩器反而一直加实例	信号把「等待」当成了「忙」	换信号，或设上限

要点 伸缩器的上限，要由下游最紧的那项资源来定

来源：L-06 第 19、20 页；L-07 第 25 页；HikariCP 文档（minimumIdle 默认等于 maximumPoolSize）

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

二、何时变

第十一讲 · 伸缩

饮 水 思 源 • 爱 国 荣 校

SJTU

何时变 · 触发

三种触发方式

方式	依据	适合	失效的时候
定时（scheduled）	已知的时间表	开放日、每周固定的高峰	时间表变了（开放推迟）；流量比预想的大
反应式（reactive）	实时指标超过目标	无法预知的突发	爬升比控制环快（第 10 页）
预测式（predictive）	历史曲线外推	有明显周期的负载	没有历史的新事件；预测错了

平台里的对应

华为云弹性伸缩（AS）：告警触发、定时触发、周期触发三类策略
Kubernetes 的 HPA：反应式

组合使用

定时打底，反应式兜底；预测式用来定「定时」开多少

要点 已知的峰要提前开，未知的峰才交给反应式

来源：华为云弹性伸缩 API 文档（伸缩策略）；Kubernetes docs (v1.37), Horizontal Pod Autoscaling

控制环的延迟



指标延迟约 20 s · 决策 ≤ 15 s · 拉镜像约 20 s · 启动预热约 60 s · 加节点约 4 min；同一比例尺

引例 • 症状 A 的时间线

9:00:00 决策：看到 8:59:40 的数据，每个 1.3 核，利用率 50%，不扩

9:00:15 决策：看到 8:59:55，每个 3.03 核，116% → 期望 4 个，新建 2 个（用现有的 2 个空位）

9:01:35：两个新实例就绪

来得及吗

反应式要来得及：预留的余量 ≥ 爬升速度 × 控制环延迟

本例：约 77 QPS/s × 100 s ≈ 7700 QPS，超过峰值 2400 本身：这个峰，反应式追不上

要点 反应式的下限是启动时间：爬升比它快的峰，靠调参追不上

来源：Kubernetes docs (v1.37), HPA；L-05 第 30 页；L-10；时间线按 l11_numbers.py 推算

何时变 · 提前

定时与预测：提前量与预测错

提前多久

= 控制环中最长的一条路径，再加余量

要加节点时：约 4 min + 80 s

开多少

按 L-10 的放置方案：6 个 Tomcat、3 个节点

而不是按伸缩器的目标值：坏一个节点也要够（L-10）

预测错了

开放推迟：多开的时间白付

流量翻倍：反应式兜底，但受数据库的上限约束（第 7 页）

回顾 L-08 第 21 页：开放日单独设 SLO 与错误预算；预热的成本与开放日的预算一起权衡

要点 定时预热用钱换时间：钱按「提前了多久、多开了多少」算

来源：L-08 第 21 页；L-10 第 27 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

三、怎么变

第十一讲 · 伸缩

饮 水 思 源 • 爱 国 荣 校

SJTU

目标追踪

期望副本数 = ⌈当前副本数 × 当前指标 ÷ 目标值⌉

与目标相差在 10% 以内不动（默认容差）

目标值按 request 计

CPU 利用率 = 用量 ÷ request。目标 70% 即每个 Tomcat 1.82 核（2.6 × 0.7），不是 limit 的 70%（2.8 核，L-07 的容量线）

例 • 引例症状 A 的第一次扩容

2 个 Tomcat，每个 3.03 核 → 利用率 116% → ⌈2 × 116 ÷ 70⌉ = 4

9:00:30 那次：新实例未就绪、按 0% 计，结果落在容差内，没有再扩

项	默认行为
决策周期	每 15 s 一次
容差	10%
扩容	不设稳定窗口；每 15 s 最多扩到当前的 2 倍或多 4 个（取大者）
缩容	稳定窗口 300 s，取窗口内最大的建议值
未就绪的实例	扩容时按 0% 计，压低扩容幅度

要点 目标追踪的公式很简单，难的是目标值和信号选什么

来源：Kubernetes docs (v1.37), Horizontal Pod Autoscaling（算法与默认 behavior）；引例按 l11_numbers.py 推算

信号选择

信号	反映什么	好处	问题
CPU 利用率	计算资源的使用	通用，不用改代码	滞后：要等实例忙起来；被 limit 封顶：request 2.6、limit 4 时最高 154%，目标 70% 下每次最多扩到约 2.2 倍；IO 型瓶颈看不出来
请求速率（QPS）	到达量	领先：爬升一开始就看得到	要知道单实例能扛多少（压测，实验 2）；请求的成本变了要重测
在途请求数	到达量 × 停留时间（Little's Law，L-06 第 16 页）	包含排队	下游变慢时停留时间变长，会误扩（第 7 页）
队列积压（lag）	异步消费者的欠账	直接反映「欠多少」	扩消费者要有可并行的分区或队列

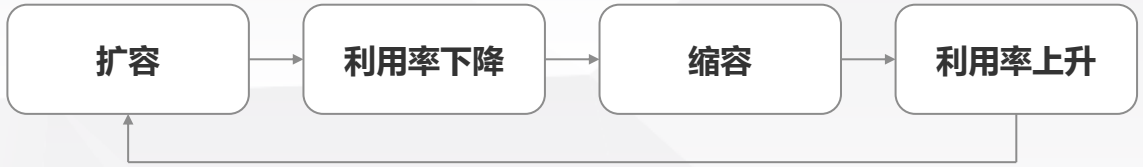
选择原则 信号要反映「被扩的这一层是否饱和」，并且领先于用户的感知

回顾 L-08 第 12 页：lag 是完成时间的先行指标

要点 信号选错，伸缩器就会在错误的层、错误的时间加实例

来源：Kubernetes docs (v1.37), HPA；L-06、L-08 讲义

稳定与缩容



抖动（flapping）：对策是缩容稳定窗口、扩快缩慢

缩容就是一次优雅停止

从负载均衡摘除 → 等在途请求完成（上限 30 s）→ 关闭连接池 → 退出（L-05 第 24 页）

代价：刚缩完又来峰值，要再付一次启动时间

正在处理长任务的实例不能随便缩：批处理要检查点（L-09 第 28 页）

节点的缩容更慢

Cluster Autoscaler（节点伸缩器）默认：

节点的 Σrequest 低于可分配量的 50%，持续 10 分钟，才删节点

加节点后 10 分钟内不评估缩容

节点上的实例要能挪到别处

按 request 判断，与调度器一致（L-10 第 5 页），不看实际用量

要点 扩要快，缩要慢；缩容的每一步都是一次实例生命周期的结束

来源：Kubernetes docs (v1.37), HPA；Kubernetes Autoscaler, Cluster Autoscaler FAQ；L-05 第 24 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

四、两层伸缩

第十一讲 · 伸缩

饮 水 思 源 • 爱 国 荣 校

SJTU

实例与节点：两个控制环

实例层 • HPA 按指标，秒到分钟

CPU 利用率 → 期望副本数 → 新建实例

没有空位时

节点层 • Cluster Autoscaler 扩容由 Pending 触发；分钟级

实例 Pending → 加节点约 4 min → 实例启动

引例：平时 2 个节点各 1 个 Tomcat，只有 2 个空位；第 5 个 Tomcat 要等加节点，9:07:20 才就绪

每个节点按内存只放 2 个 Tomcat（L-10）：第 3 个起，控制环就从约 2 min 变成约 6 min

两层各管什么

实例层决定快不快

节点层决定省不省

节点伸缩器按 request 判断，每 10 s 扫描一次；新节点多久可用取决于云平台（FAQ：GCE 上通常 3 ~ 4 分钟）

钱按节点算

只缩实例不缩节点，空出来的只是 request 的账面空间

除非有批处理用掉它（L-10 第 8 页：形状互补的负载混放）

要点 实例层决定快不快，节点层决定省不省

来源：Kubernetes Autoscaler, Cluster Autoscaler FAQ；L-10 第 8、27 页；引例按 l11_numbers.py 推算

纵向伸缩与其他方式

方式	改什么	反应	适合	代价
水平（HPA）	副本数	秒到分钟	无状态服务	要无状态；要有连接预算
纵向（VPA、原地调整资源）	单个实例的 request、limit	分钟	难以水平扩的组件；request 定得不准的服务	单个实例的上限是节点大小；原地调整资源 v1.35 起为正式功能，VPA 的原地更新模式为 beta
云主机伸缩组（如华为云 AS）	云主机台数	分钟	没有容器编排的部署	粒度粗：整台云主机
Serverless	调用的并发	毫秒到秒，可缩到零	低频接口（L-10 第 20 页）	冷启动；开放日的突发不适合

选课平台 Tomcat 水平伸缩；MySQL 不进弹性层，规格调整按计划停机或主从切换（模块 7）

要点 水平扩副本，纵向调规格；有状态组件的伸缩常常是提前计划好的变更

来源：Kubernetes Blog, In-Place Pod Resize Graduates to Stable (2025-12-19)；华为云弹性伸缩文档；L-10 第 20 页

上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



互联网产品开发与运维

五、课程实践：弹性方案

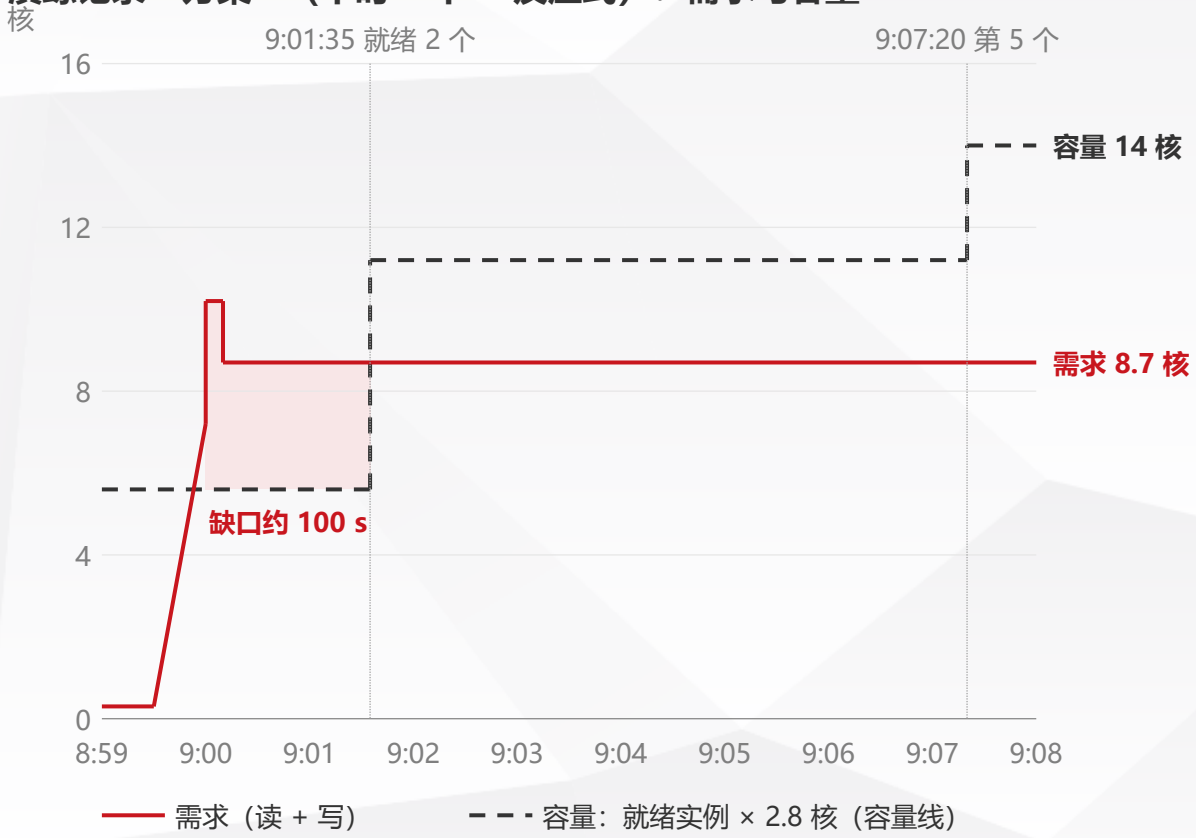
第十一讲 · 伸缩

饮 水 思 源 • 爱 国 荣 校

SJTU

参数设定与演练记录

演练记录 • 方案 B（平时 2 个 + 反应式）：需求与容量



沿用

L-09 配额：Tomcat request 2.6 核、limit 4 核；L-10 放置：每节点 2 个 Tomcat，开放日 6 个、3 个节点；L-07：读 3 ms、写 5 ms，写上限约 250 TPS

新增（本讲设定）

读流量 8:59:30 起 30 s 内 100 → 2400 QPS，维持到 9:10，9:30 降到约 300；写 9:00 起前 10 s 600 TPS，之后 300

平时 2 个节点、2 个 Tomcat；伸缩器按 CPU，目标 70%，2 ~ 6 个；指标延迟约 20 s

拉镜像 20 s、启动预热 1 min；加节点约 4 min，每个新节点 2 个空位

连接池每个 Tomcat 20；批处理 10、其他约 5；max_connections 151

一学期 120 天，开放日 3 天；按节点·小时计费

注 数值为设定，与 L-07 的资源账、L-09 的配额表、L-10 的放置方案自洽

来源：L-07 第 24、25 页；L-09 第 29 页；L-10 第 27 页；曲线按 l11_numbers.py 推算

课堂讨论

1 赶不上

方案 B 的缺口从何时到何时？控制环的延迟由哪几段组成？

把目标调低、决策周期调短，能消除这个缺口吗？

2 提前开

方案 C 定时把最少副本数调到 6（6 个 Tomcat、3 个节点），最晚几点开始？

比常驻 3 个节点（方案 A）一学期省多少节点·小时？开放推迟或流量翻倍时怎么办？

3 扩到哪儿为止

有人把目标调到 50%、上限调到 10，扩到 7 个 Tomcat 时出了什么事？

上限应该定多少？提交变慢，靠伸缩能解决吗？

4 什么时候缩

开放日过后，Tomcat 和节点各在什么时候缩？

怎么缩才不丢请求？

安排 每题小组讨论 2 分钟、汇报 1 分钟，其余时间讲评

讨论要点（1） （2）

1 赶不上：启动时间压不掉

缺口：2 个 Tomcat 按容量线承担 5.6 核，8:59:53 起需求超过它；9:00～9:01:35 需求 10.2、8.7 核，超过 2 个的 limit 总和 8 核，请求排队；9:01:35 两个新实例就绪，缺口结束，约 100 s

延迟：指标 20 s + 等下一次决策 ≤ 15 s + 拉镜像 20 s + 启动预热 60 s；要加节点再加 4 min

调参只能压缩前两段；启动的 80 s 压不掉，爬升却只有 30 s

出路：把启动做快（L-05 第 24 页、预拉镜像），或者提前开

2 提前开：用钱换时间

最晚：9:00:00 −（4 min + 80 s）= 8:54:40；要在 8:59:30 爬升前就绪，则为 8:54:10。取 8:50，留出余量

方案 A：3 × 24 × 120 = 8640 节点·小时

方案 C：平时 2 个节点，开放日多 1 个（8:50～约 10:10，1.33 h）：2 × 24 × 120 + 3 × 1.33 ≈ 5764，省约 33%

开放推迟：定时任务跟着开放时间走，多开的时间照付；流量翻倍：反应式兜底，但上限受第 3 题的连接预算约束

结论 爬升快于启动的峰，只能提前开；提前开的钱按节点·小时算

来源：按 l11_numbers.py 推算

讨论要点（3） （4）

3 扩到哪儿为止：由下游定

可给 Tomcat 的连接：151 − 10 − 5 = 136；每个 20 → 最多 6 个（120）。第 7 个一启动就要 20 个：140 + 15 = 155 > 151，报 Too many connections

上限：6 个，与 L-10 的放置方案一致；不是越多越安全

提交：写的上限在磁盘，约 250 TPS（L-07 第 25 页）。多出的 Tomcat 只带来更多并发事务和锁等待，p99 更差

对策在写路径本身：入口按 250 TPS 限流、组提交（实验 2）、Redis 原子扣减名额（思考题 2 的做法）

4 什么时候缩：慢一点

Tomcat：9:30 最少副本数改回 2，此时读流量约 300 QPS（约 1 核）；按 300 s 稳定窗口，约 9:35 缩到 2 个，每个按优雅停止退出

节点：Σrequest 低于 50% 持续 10 分钟才删；批处理 10:00 结束前，它所在的节点不空

为何定时调的是「最少副本数」：峰值时按目标 70% 只需 5 个（8.7 ÷ 1.82 = 4.8），否则过了稳定窗口会缩掉第 6 个，坏一个节点就不够了

结论 伸缩的上限由下游定，开放日的下限由放置方案定

来源：L-06 第 19 页；L-07 第 25 页；Kubernetes HPA 与 Cluster Autoscaler 的默认值

资源管理 – 伸缩

参考弹性方案



护栏	取值	依据
最少副本数	平时 2；开放日 8:50 ~ 9:30 为 6	平时坏一个节点仍有 1 个；开放日按 L-10 的放置
最多副本数	6	MySQL 的连接预算（第 3 题）
伸缩信号	CPU 70%，作为兜底；开放日以定时为主	第 1 题
缩容	稳定窗口 300 s（默认）；开放日由最少副本数挡住	第 4 题
写入口	限流 250 TPS	磁盘的上限（L-07）

结论 弹性方案 = 定时打底 + 反应式兜底 + 由下游定的上限；预热时间与上限都要在演练里验证

来源：按 l11_numbers.py 推算；节点删除时间按偏保守估计

本讲小结

- **扩哪一层：瓶颈在被扩的层、且无状态，伸缩才有效**
  - 上限由下游最紧的资源定
- **何时变：已知的峰提前开**
  - 反应式的下限是启动时间
- **怎么变：目标按 request 计，信号要反映被扩的层**
  - 扩快缩慢；缩容就是优雅停止
- **两层伸缩：实例层决定快不快，节点层决定省不省**
  - 钱按节点算

判断标准 这份容量什么时候需要、多久能到位？扩上去以后，下一个到上限的是谁？

模块 6 总结：三问加一个保证

L-09 隔离与分配

给多少：配额表

判断标准

这份资源给谁、给多少？不够或超了，谁受损、代价是什么？

L-10 调度

放哪儿：放置方案

判断标准

这份资源放在哪儿？坏了一处，丢多少、多久补回、谁让位？

L-11 伸缩

何时变：弹性方案

判断标准

这份容量什么时候需要、多久能到位？扩上去以后，下一个到上限的是谁？

三张表从哪里来 都由同一份资源账（L-07）和 SLO（L-08）推出；隔离保证给的就是拿到的（L-09 第 5 页）

引出模块 7 三讲里反复出现「期望状态」和「控制环」（L-10 第 21 页）。模块 7 从应用的角度看同一件事：声明期望状态（IaC）、健康检查与自愈、发布与配置、故障处置

要点 模块 6 回答资源怎么分、怎么放、怎么变；模块 7 回答运行中的应用怎么管

来源：L-09 第 5 页；L-10 第 21 页；L-08 第 33 页（模块 5 总结）

实验 3 预告

方向一：节流与 OOM

在自己的主机上，用 cgroup 限制示例后端，复现 CPU 节流与 OOM

读计数器，与 L-09 的配额算术对照

方向二：伸缩控制环

实现或配置一个伸缩控制环，在开放日负载下比较常驻、反应式、定时预热的缺口、p99 与实例·分钟

用实测的各段延迟，替换本讲的设定

安排 10/30 发布，11/20 检查截止，含答辩；另一半是模块 7 的内容

具体任务、环境与评分以实验 3 指导书为准

要点 思考题用给定的数据推理，实验用自己测到的数据验证

课后思考与参考资料

课后思考

1. 为小组项目画一条控制环延迟的时间条：指标延迟、决策周期、拉镜像、启动、就绪各多久（实测或估算）？负载爬升有多快?

2. 小组项目的数据库 max_connections 是多少？按每个实例的连接池算，应用最多能扩到几个实例?

3. 小组项目有没有「已知的峰」？有，写出定时预热的时间与规模；没有，反应式的信号选什么?

参考资料

Kubernetes docs：Horizontal Pod Autoscaling；Vertical Pod Autoscaling；Resize CPU and Memory Resources assigned to Containers

Kubernetes Autoscaler：Cluster Autoscaler FAQ

HikariCP：About Pool Sizing

Google SRE Book：Ch. 21 Handling Overload；Ch. 22 Addressing Cascading Failures

华为云文档：弹性伸缩（伸缩策略）

感谢聆听

饮 水 思 源　爱 国 荣 校